COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 1, 2026	**DATE SUBMITTED:**	October 6, 2026
NAME:	Santiago, Niña Denise M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY NO. 10.1 **Graphs**

1. PROCEDURE OUTPUT

PROCEDURE ILOA – Graph Implementation Utilizing Adjacency Matrix and Adjacency List

ILOA10p1.cpp > ...

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {

    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;

        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }

    int N; // number of nodes in the graph

public:

    adjNode** head; // adjacency list as array of pointers
```

```
33      // Constructor
34      DiaGraph(graphEdge edges[], int n, int N) {
35
36          // allocate new node
37          head = new adjNode*[N];
38          this->N = N;
39
40          // initialize head pointer for all vertices
41          for (int i = 0; i < N; ++i)
42              head[i] = nullptr;
43
44          // construct directed graph by adding edges to it
45          for (unsigned i = 0; i < n; i++) {
46
47              int start_ver = edges[i].start_ver;
48              int end_ver = edges[i].end_ver;
49              int weight = edges[i].weight;
50
51              // insert in the beginning
52              adjNode* newNode =
53                  getAdjListNode(end_ver, weight, head[start_ver]);
54
55              // point head pointer to new node
56              head[start_ver] = newNode;
57          }
58      }
```

```
60      // Destructor
61      ~DiaGraph() {
62
63          for (int i = 0; i < N; i++) {
64
65              adjNode* current = head[i];
66
67              while (current != nullptr) {
68                  adjNode* temp = current;
69                  current = current->next;
70                  delete temp;
71              }
72          }
73
74          delete[] head;
75      }
76  };
```

```cpp
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i)
{
    while (ptr != nullptr) {

        std::cout << "(" << i << ", "
                  << ptr->val << ", "
                  << ptr->cost << ") ";

        ptr = ptr->next;
    }

    std::cout << std::endl;
}
```

```cpp
// display adjacency matrix
void display_AdjMatrix(graphEdge edges[], int n, int N)
{
    int matrix[10][10] = {0};

    // place edge weights into the matrix
    for (int i = 0; i < n; i++) {

        int start = edges[i].start_ver;
        int end = edges[i].end_ver;
        int weight = edges[i].weight;

        matrix[start][end] = weight;
    }

    std::cout << "\nGraph adjacency matrix\n";
    std::cout << "(0 means there is no edge)\n\n";

    std::cout << "    ";

    for (int i = 0; i < N; i++)
        std::cout << i << " ";

    std::cout << std::endl;

    for (int i = 0; i < N; i++) {

        std::cout << i << "   ";

        for (int j = 0; j < N; j++)
            std::cout << matrix[i][j] << " ";

        std::cout << std::endl;
    }
}
```

```cpp
// graph implementation
int main()
{
    // graph edges array
    graphEdge edges[] = {

        // (x, y, w) -> edge from x to y with weight w

        {0, 1, 2},
        {0, 2, 4},
        {1, 4, 3},
        {2, 3, 2},
        {3, 1, 4},
        {4, 3, 3}
    };

    int N = 6; // Number of vertices in the graph

    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // display adjacency matrix
    display_AdjMatrix(edges, n, N);

    // display adjacency list representation of graph
    std::cout << "\nGraph adjacency list"
              << std::endl;

    std::cout << "(start_vertex, end_vertex, weight):"
              << std::endl;

    for (int i = 0; i < N; i++) {

        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

CODE OUTPUT

```
Graph adjacency matrix
(0 means there is no edge)

    0 1 2 3 4 5
0   0 2 4 0 0 0
1   0 0 0 0 3 0
2   0 0 0 2 0 0
3   0 4 0 0 0 0
4   0 0 0 3 0 0
5   0 0 0 0 0 0

Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

PROCEDURE ILO B.1 & B.2 – Graph Implementing Graph Traversal Algorithms (Breadth-First and Depth First Search)

graph.h

C graph.h > ...

```h
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge {
    size_t src, dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const {
        return weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const {
        return weight > e.weight;
    }
};
```

```h
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
    for (size_t i = 1; i <= G.vertices(); i++) {
        os << i << ":\t";
        for (const auto &e : G.outgoing_edges(i))
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

```cpp
template <typename T>
class Graph {
public:
    Graph(size_t N) : V(N) {}

    size_t vertices() const {
        return V;
    }

    const std::vector<Edge<T>> &edges() const {
        return edge_list;
    }

    void add_edge(Edge<T> &&e) {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    std::vector<Edge<T>> outgoing_edges(size_t v) const {
        std::vector<Edge<T>> result;
        for (const auto &e : edge_list)
            if (e.src == v)
                result.emplace_back(e);
        return result;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os,
                                    const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};
```

```cpp
// Create the graph from the activity
template <typename T>
Graph<T> create_reference_graph() {
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (const auto &i : edges)
        for (const auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}
```

```cpp
// Depth-First Search
template <typename T>
std::vector<size_t> depth_first_search(const Graph<T> &G, size_t dest) {
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1);

    while (!stack.empty()) {
        size_t current = stack.top();
        stack.pop();

        if (visited.find(current) == visited.end()) {
            visited.insert(current);
            visit_order.push_back(current);

            for (const auto &e : G.outgoing_edges(current))
                if (visited.find(e.dest) == visited.end())
                    stack.push(e.dest);
        }
    }
    return visit_order;
}
```

```cpp
// Breadth-First Search
template <typename T>
std::vector<size_t> breadth_first_search(const Graph<T> &G, size_t dest) {
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1);

    while (!queue.empty()) {
        size_t current = queue.front();
        queue.pop();

        if (visited.find(current) == visited.end()) {
            visited.insert(current);
            visit_order.push_back(current);

            for (const auto &e : G.outgoing_edges(current))
                if (visited.find(e.dest) == visited.end())
                    queue.push(e.dest);
        }
    }
    return visit_order;
}

#endif
```

main.cpp

main.cpp > ...

```cpp

// Test DFS
void test_DFS() {
    auto G = create_reference_graph<T>();

    std::cout << "===================== GRAPH =====================" << std::endl
    std::cout << G << std::endl;

    std::cout << "=============== DEPTH-FIRST SEARCH ===============" << std::end
    std::cout << "DFS Order of vertices:" << std::endl;

    auto dfs_visit_order = depth_first_search(G, 1);

    for (auto v : dfs_visit_order)
        std::cout << v << " ";

    std::cout << "\n\n";
}

// Test BFS
void test_BFS() {
    auto G = create_reference_graph<T>();

    std::cout << "============== BREADTH-FIRST SEARCH ==============" << std::end
    std::cout << "BFS Order of vertices:" << std::endl;

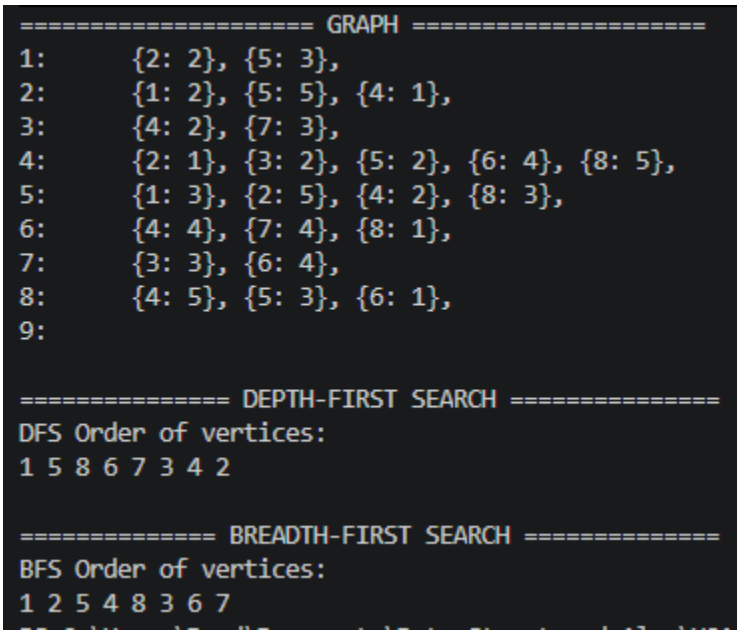
    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order)
        std::cout << v << " ";

    std::cout << std::endl;
}

int main() {
    test_DFS();
    test_BFS();
    return 0;
}
```

CODE OUTPUT



CODE ANALYSIS:

ILO A: Graph Implementation

In ILO A, the program focuses on creating and displaying a graph through the DiaGraph class. The code uses an adjacency list and an adjacency matrix to represent the connections between vertices. Functions such as display_AdjList() and display_AdjMatrix() are responsible for showing the graph in two different formats. The program also uses structures to store node and edge information, while pointers and dynamic memory allocation help manage the linked list. The destructor handles the removal of allocated memory after it is used. These parts of the code work together to build the graph and display its connections correctly.

ILO B: DFS and BFS

In ILO B, the program focuses on exploring the vertices of the graph using DFS and BFS. The create_reference_graph() function sets up the graph, while test_DFS() and test_BFS() run the two traversal algorithms. DFS uses a stack to explore one path before returning to other available vertices, whereas BFS uses a queue to process vertices level by level. Both algorithms use a set to track visited vertices and prevent repeated visits. The program then displays the order in which the vertices are explored, starting from vertex 1. This implementation demonstrates how different data structures and functions control the traversal process and produce different visiting orders.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOC – Graph Implementation, Operations and Traversal Methods

1. **A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore another vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.**

- The most suitable algorithm for the given situation is Depth-First Search (DFS). As it visits a vertex and follows one path as far as possible before going back to the previous vertex to explore other paths. For example, if a person starts at Location A and travels through Locations B and C, the person continues along that route until there are no more unvisited locations. After that, the person goes back to the previous location and checks for other paths to explore. This process continues until all reachable locations have been visited. DFS is the best choice because it follows the idea of exploring one path first, then backtracking to discover other possible paths. In simpler terms, it is like landmarking in Hansel and Gretel.

2. **Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.**

- The equivalent of Depth-First Search (DFS) in tree traversal is Depth-First Tree Traversal, which can be performed using Preorder, Inorder, or Postorder Traversal. These methods explore a branch of the tree before moving to another branch. However, their visiting order is different. Preorder visits the root first, Inorder visits the root between the left and right subtrees, and Postorder visits the root last.

Method	Visiting Order
Depth-First Search (DFS)	Start → Go Deeper → Backtrack → Explore Next
Preorder Traversal	Visit Root → Visit Left → Visit Right
Inorder Traversal	Visit Left → Visit Root → Visit Right
Postorder Traversal	Visit Left → Visit Right → Visit Root

3. **In the performed code, what data structure is used to implement the Breadth First Search?**

- The data structure used in Breadth-First Search (BFS) is a queue. In the code, the queue holds the vertices that the algorithm needs to visit. When a vertex is visited, its unvisited neighbors are added to the queue. The queue follows the First-In, First-Out (FIFO) rule, which means the vertex added first will be processed first. This allows BFS to visit the nearby vertices before moving to the next level of the graph. Using a queue helps the program explore the graph in order and visit each reachable vertex without repeating the same vertex.

4. **How many times can a node be visited in the BFS?**

- In Breadth-First Search (BFS), a node can be visited only once. In the code, the visited set keeps track of the nodes that have already been visited. Before processing a node, the program checks whether it has already been visited. If it has, the program skips it to avoid repeating the same work. This is important because a node may be connected to several other nodes, so the algorithm might encounter it more than once. By keeping track of visited nodes, BFS can explore the graph properly without processing the same node repeatedly.

CONCLUSION / LEARNINGS / ANALYSIS

Throughout this module, I learned how graphs can be implemented to represent connections between different vertices. I learned how to use an adjacency list and an adjacency matrix to store and display the connections and weights of a graph. I also gained a better understanding of how structures, classes, pointers, linked lists, and arrays work together in creating a graph. Furthermore, I learned how Depth-First Search (DFS) and Breadth-First Search (BFS) explore the vertices in different ways. DFS uses a stack to explore one path deeply before going back to other paths, while BFS uses a queue to visit nearby vertices first. I also understood the importance of using a set to keep track of visited vertices and avoid visiting the same vertex repeatedly. Overall, this activity made me realize that the lessons from our previous modules are all connected and serve as the foundation for learning more advanced topics in Data Structures and Algorithms. The concepts and skills we learned earlier are important because they help us understand and implement new algorithms more easily.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ***Grammarly and Microsoft Copilot***

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: ***Grammar correction*** **and Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Since I was absent during this activity, I used Microsoft Copilot to help me better understand the code and concepts that lie within the program from the Procedure part. While, I used Grammarly to improve my explanation grammar and vocabulary. Despite this help, the AI services used did not replace original thinking, analysis, authorship, or completion of the required work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.